
Proyecto Taxalia

Web corporativa + asistente conversacional Lexi

Informe técnico

Decisiones, tecnología y el sistema multiagente

Proyecto: Taxalia (firma de asesoría)
Arquitectura: Meta-repo: frontend Astro + backend de chatbot
Agente: Lexi — orquestador multiagente sobre Ollama
Fecha: Junio de 2026

Índice

1. Resumen ejecutivo	2
1.1. Cifras del proyecto	2
2. Contexto y objetivos	2
3. Arquitectura general	2
3.1. Arquitectura “screaming”	3
4. Decisiones tomadas	3
4.1. Decisiones de estructura (ADR)	3
4.2. Decisiones del agente (SDD)	3
4.3. Decisión de entrega: PRs encadenados	4
5. Stack tecnológico	4
5.1. Frontend	4
5.2. Backend del agente	5
6. El agente Lexi: el corazón del proyecto	5
6.1. Personas como ficheros en disco	5
6.2. El pipeline de una conversación	5
6.3. Ensamblado del system prompt por capas	6
6.4. Las 5 políticas de conducta	6
6.5. Observabilidad y manejo de fallos	6
7. Calidad: pruebas y presupuestos	7
7.1. Estrategia de pruebas	7
7.2. Presupuestos de rendimiento	7
8. Despliegue	7
9. Estado actual y próximos pasos	7
10. Conclusión	8

1 Resumen ejecutivo

Taxalia es la web corporativa de una firma de asesoría profesional (consultoría, valoración y acompañamiento estratégico). El proyecto va más allá de un sitio estático: incorpora **Lexi**, un asistente conversacional propio construido como un **sistema multiagente** que se ejecuta contra un modelo de lenguaje local (Ollama) y se despliega también en la nube.

La pieza diferencial no es “un chatbot” sino la **ingeniería que lo sostiene**: un orquestador que decide qué agentes responden, ejecución en paralelo con cancelación cooperativa, un sintetizador que funde varias respuestas en una sola, políticas de conducta versionadas en disco y una estrategia de pruebas y observabilidad de nivel producción.

En una frase: una web ligera en Astro con coste de dependencias casi nulo, y detrás un backend de agentes en TypeScript estricto, diseñado con metodología *Spec-Driven Development* (SDD), decisiones documentadas como ADR y un contrato HTTP/SSE que desacopla por completo el front del motor de IA.

1.1 Cifras del proyecto

Indicador	Valor
Código fuente del backend (TypeScript)	~2.655 líneas
Ficheros fuente del backend	36 módulos
Suites de pruebas del backend	22 ficheros (unitarias + integración)
Código fuente del frontend	~3.220 líneas
Agentes especializados	3 (advisory, valuation, financial)
Políticas de conducta	5 (cargadas en arranque)
Capacidades (skills)	3
Nuevas dependencias npm en el frontend	0
Decisiones de arquitectura formales	2 ADR + 6 SDD

2 Contexto y objetivos

La web de Taxalia se diseñó para presentar la identidad de la empresa, mostrar sus servicios, publicar contenidos de blog y facilitar el contacto, con soporte multilingüe (inglés y español). El sitio es estático, ligero y fácil de desplegar.

El punto de partida tenía una limitación clara: el *widget* de chat era un ancla estática que redirigía a la página de contacto. El objetivo del trabajo fue **sustituir ese marcador por una superficie conversacional real**, sin comprometer la simplicidad del sitio: sin integraciones de Astro, sin cambios en el *pipeline* de build y **sin añadir una sola dependencia npm al frontend**.

La restricción de diseño “*cero dependencias nuevas en el front*” obligó a una solución limpia: el cliente de chat usa solo APIs nativas del navegador (`fetch`, `ReadableStream`, `AbortController`, `crypto.randomUUID`). Toda la complejidad vive en un backend independiente, conectado por un contrato HTTP.

3 Arquitectura general

El repositorio es un **meta-repo** con dos proyectos hermanos y autónomos, más la tooling transversal:

```
taxalia/ <- meta-repo (sin package.json propio)
|-- docs/adr/ <- decisiones de arquitectura (ADR)
|-- openspec/ <- artefactos SDD (propuesta, diseno, specs, tareas)
```

```

|-- scripts/setup.mjs <- bootstrap de ambos lados (cross-platform)
|-- setup.sh <- wrapper Unix delgado
|-- frontend/ <- sitio Astro (submodulo Git)
| |-- src/components/ ChatWidget.astro, Header, Hero, Services...
| |-- src/scripts/ chat-client.ts (consumidor SSE)
| '-- src/pages/ index, services, blog, contact, /es/...
'-- backend/ <- chatbot multiagente (submodulo Git)
    '-- src/{chat,agents,skills,conducta,dispatch,ollama,observability}

```

Cada lado es independiente: `cd frontend && npm install && npm run dev` funciona sin asumir nada de la raíz, y el backend declara su propia configuración de TypeScript sin depender de ficheros del front.

3.1 Arquitectura “screaming”

El backend aplica *screaming architecture*: los nombres de carpeta gritan el dominio, no el framework. Las carpetas (`chat`, `agents`, `skills`, `conducta`, `dispatch`, `ollama`, `observability`) sobreviven a cualquier cambio de tecnología: si se sustituye Hono, `chat/` permanece; si se sustituye Ollama, `ollama/` pasa a llamarse `inference/`. El dominio manda sobre la herramienta.

4 Decisiones tomadas

Las decisiones no son implícitas: están documentadas como **ADR** (Architecture Decision Records) y como resoluciones dentro del flujo **SDD**. Esto deja una traza auditable del *por qué*, no solo del *qué*.

4.1 Decisiones de estructura (ADR)

ADR	Decisión	Razón
0001 — Backend hermano	Mantener backend como carpeta hermana <code>backend/</code> , sin npm workspaces.	Un solo servicio; se difiere la tooling de monorepo hasta que aparezca un segundo backend o un paquete compartido.
0002 — Frontend agrupado	Mover el proyecto Astro a <code>frontend/</code> ; la raíz queda como meta-repo puro sin código de aplicación.	Separa la app de la tooling transversal; rompe el acoplamiento oculto del <code>tsconfig</code> del backend hacia el front.

Ambos ADR fijan el mismo **disparador de migración** a npm workspaces: cuando exista un segundo backend o un paquete compartido por ambos lados. Hasta entonces, la forma multirepo es la correcta.

4.2 Decisiones del agente (SDD)

El sistema multiagente se diseñó con *Spec-Driven Development*: primero exploración, luego propuesta, especificaciones, diseño y tareas, antes de escribir código. Las decisiones clave resueltas:

Cuestión	Decisión	Razón
Agentes v1	advisory, valuation, financial.	Alinea los agentes con la taxonomía real de servicios de la web.
Conducta v1	5 políticas: no fingir, citar fuentes, respuesta bilingüe, sin PII, traspaso a humano.	Cubre la superficie de riesgo v1: alucinación, citación, idioma, privacidad y escalado.
Concurrencia	Tope de 2 <i>dispatches</i> simultáneos vía semáforo en proceso.	Un equipo M3 de 16 GB se satura con 2 dispatches; más invita a OOM.
Reintentos	Sin reintentos automáticos ante fallo transitorio; <i>fail loud</i> con aviso.	Los reintentos silenciosos en un modelo local pequeño producen UX confusa.
Autenticación	Anónimo + lista blanca CORS.	El backend corre en local sobre Ollama; sin exposición pública en v1.
Estimador de tokens	Heurística calibrada (4 chars/token + sobrecarga).	Validada contra <code>prompt_eval_count</code> real de Ollama, error $\leq 15\%$.

4.3 Decisión de entrega: PRs encadenados

El trabajo se planificó como **5 pull requests encadenados**, cada uno con un presupuesto de revisión ≤ 400 líneas, para mantener la revisión enfocada: (1) higiene + setup, (2) esqueleto Hono + contrato SSE, (3) cargadores + ensamblado del system prompt, (4) orquestador + dispatch paralelo + sintetizador, (5) reemplazo del widget y cableado SSE en el front. Cada PR define su propio plan de *rollback*.

5 Stack tecnológico

5.1 Frontend

Tecnología	Rol
Astro 6.3	Generación de sitio estático, ligero y fácil de desplegar.
TypeScript	<code>chat-client.ts</code> : consumidor SSE y enlaces al DOM.
i18n manual	Estrategia propia (<code>/es/</code> + <code>i18n.ts</code>), sin <code>@astrojs/i18n</code> .
CSS propio	<code>lb-co.css</code> ; estética corporativa dorado/azul.
APIs nativas	<code>fetch</code> , <code>ReadableStream</code> , <code>AbortController</code> .

5.2 Backend del agente

Tecnología	Rol
Node 24 + TypeScript estricto	Runtime y tipado fuerte de extremo a extremo.
Hono	Framework HTTP que sirve la superficie de chat y el streaming SSE.
Zod	Esquemas como única fuente de verdad (request, SSE, defs de ficheros).
Ollama (ollama-js)	Cliente del modelo; gemma4:e4b en local, gemma4:31b-cloud en producción.
Pino	Logging estructurado con <code>requestId</code> por petición.
Vitest	Pruebas unitarias y de integración (proyectos separados).

El **contrato de tipos** se define una sola vez con Zod y se infiere con `z.infer<...>`, de modo que el esquema de validación en runtime y el tipo de TypeScript nunca divergen.

6 El agente Lexi: el corazón del proyecto

Aquí está el verdadero valor. “Lexi” no es una llamada a un LLM con un prompt: es un **sistema multiagente** con orquestación, paralelismo, síntesis, políticas y observabilidad. Estos son sus componentes.

6.1 Personas como ficheros en disco

Agentes, capacidades y conducta son **ficheros Markdown con frontmatter YAML**. Cambiar el comportamiento del asistente no requiere tocar código: basta editar un fichero y reiniciar (o llamar a `POST /admin/reload` en desarrollo).

```

---
id: advisory
name: Advisory Services Assistant
description: Helps users understand Taxalia advisory services.
system_prompt: |
  You are Taxalia's advisory services assistant. Explain
  engagement models clearly in English or Spanish...
tools: [lookup-engagement-model, capture-lead]
tags: [advisory, services]
---
```

Esta separación *datos vs. código* es lo que convierte al asistente en un producto mantenible por personas no programadoras.

6.2 El pipeline de una conversación

Cada mensaje del usuario atraviesa cuatro etapas claramente separadas:

- Orquestación** — una llamada al modelo en formato JSON decide *qué* agentes deben responder (`agentsToRun`). Saludos o intención desconocida producen un array vacío. Nunca se inventan agentes.
- Dispatch paralelo** — los agentes seleccionados se ejecutan a la vez (`Promise.allSettled`), cada uno con timeout de 30s y su propio `AbortController` enlazado a la señal externa.

3. **Síntesis** — si responden 2 o más agentes, un sintetizador funde sus salidas en una única respuesta coherente.
4. **Streaming SSE** — la respuesta vuelve al navegador token a token como *Server-Sent Events*; el widget va componiendo la burbuja.

Cancelación cooperativa de extremo a extremo. Si el usuario cierra el chat, el `AbortController` del navegador propaga la señal a la ruta de Hono, de ahí a cada `AbortController` por agente y de ahí al *stream* de Ollama. Todo se cancela en ≤ 200 ms. Es el tipo de detalle que separa un prototipo de un sistema bien construido.

6.3 Ensamblado del system prompt por capas

El prompt de sistema se construye en un orden **deliberado**, de lo más general a lo más específico: conducta → persona del agente → metadatos de skills → historial. Este orden no es estético: es una defensa empírica frente a *prompt injection*. Además, si el prompt supera 1.500 tokens, se rechaza con error explícito en lugar de degradar en silencio.

6.4 Las 5 políticas de conducta

Las políticas se cargan en arranque y se concatenan en *cada* prompt de agente. El arranque **verifica que sean exactamente 5** (una sexta haría fallar el boot, a propósito: cada política debe ser una incorporación consciente).

Política	Regla
never-pretend	Decir “no lo sé” antes que inventar hechos, precios o conclusiones legales.
cite-sources	Citar una página de servicio de Taxalia o admitir que no hay fuente.
bilingual-response	Responder en el idioma del usuario; no mezclar.
privacy-no-pii	No almacenar, repetir ni inventar datos personales.
handoff-to-human	Si piden un humano, ofrecer el enlace de contacto.

6.5 Observabilidad y manejo de fallos

El sistema es transparente sobre lo que ocurre:

- **Trazabilidad:** cada petición lleva un `requestId` (cabecera `X-Request-Id` o UUID v4 generado) que viaja por todos los logs y vuelve en el evento final del SSE.
- **Métricas:** contadores en memoria expuestos en `GET /metrics` (chats iniciados/completados, timeouts por agente, fallos parciales, errores de parseo del orquestador...).
- **Fallo parcial con aviso:** si un agente falla, su error se reporta en `done.agents[]` y el sintetizador trabaja con los que sí respondieron; el usuario ve un aviso de “respuesta posiblemente incompleta”. Nunca se oculta un fallo.
- **Privacidad por diseño:** los logs registran `requestId`, agentes, idioma, conteo de tokens y latencia, **nunca** el contenido de los mensajes.

7 Calidad: pruebas y presupuestos

7.1 Estrategia de pruebas

22 suites cubren las dos capas. Lo destacable es *qué* se prueba:

- **Pureza del system prompt:** se ensambla 1.000 veces y se verifica que el resultado es idéntico byte a byte.
- **Carrera de cancelación:** un cliente sintético abre la conexión, espera el primer evento y aborta; se afirma que todos los `AbortController` disparan en ≤ 200 ms.
- **Tasa de parseo del orquestador:** el modelo real se enfrenta a 20 mensajes de prueba; criterio de aprobado $\geq 16/20$ con JSON parseable.
- **Arranque en frío:** la primera petición contra un Ollama recién levantado responde dentro de la ventana de gracia de 60 s.

7.2 Presupuestos de rendimiento

Las metas no son aspiracionales: están escritas y testeadas.

Métrica	Presupuesto
GET /health (sin llamar a Ollama)	≤ 100 ms
Gracia de arranque en frío (una vez por proceso)	60 s
Timeout por agente	30 s
System prompt ensamblado	≤ 1.500 tokens
Propagación del abort (cliente $\rightarrow$ Ollama)	≤ 200 ms
Primer token con 2 agentes (en caliente)	≤ 8 s
Respuesta completa, 2 agentes (≤ 200 tokens)	≤ 30 s

8 Despliegue

El sistema funciona en dos modos sin cambios de código, solo de configuración:

- **Local:** Ollama en 127.0.0.1:11434 con el modelo `gemma4:e4b`. El script `setup.sh / scripts/setup.mjs` es idempotente: comprueba Ollama, descarga el modelo y ejecuta `npm ci` en ambos lados.
- **Producción:** backend como función *serverless* en Vercel (`vercel.json` incluye los ficheros de agentes/skills/conducta en el bundle), apuntando al modelo en la nube `gemma4:31b-cloud`.

La selección de host y modelo según `NODE_ENV` está centralizada en `config.ts` y validada con Zod, de modo que un entorno mal configurado falla en el arranque con un mensaje claro, no a mitad de una petición.

9 Estado actual y próximos pasos

El núcleo conversacional está operativo de extremo a extremo: orquestación, dispatch paralelo, síntesis, streaming, conducta, observabilidad, pruebas y despliegue dual. El frontend ya integra el widget real con avatar, redimensionado y página de *health check*.

Quedan tareas de producto recogidas en el *backlog* del proyecto, entre ellas: completar páginas internas (servicios individuales, blog con *Content Collections*), conectar el formulario de contacto a un backend real, cerrar la internacionalización al español, y reforzar SEO/accesibilidad (sitemap, `robots.txt`, Open Graph, JSON-LD, navegación por teclado).

10 Conclusión

Taxalia demuestra que se puede añadir IA conversacional a un sitio estático **sin sacrificar su simplicidad ni inflar sus dependencias**. El valor no está en “tener un chatbot”, sino en cómo está construido:

- Un **sistema multiagente** real —orquestador, paralelismo, sintetizador— en lugar de una llamada suelta a un LLM.
- **Separación datos/código:** personas y políticas como ficheros editables sin tocar el motor.
- **Robustez de producción:** cancelación cooperativa, fallo parcial con aviso, presupuestos de rendimiento testeados y observabilidad con `requestId` de punta a punta.
- **Disciplina de ingeniería:** decisiones documentadas (ADR + SDD), TypeScript estricto, contrato de tipos único con Zod y entrega en PRs encadenados revisables.

El agente montado es la mejor carta de presentación técnica del proyecto: un componente pequeño en líneas de código pero grande en criterio de ingeniería, donde cada decisión está justificada y cada modo de fallo está previsto.